

Checklist:

Team Topologies – Team-First Boundaries

1. Start with the team, not the architecture

- ☐ Is this boundary small enough for one team to understand, own, and evolve?
- ☐ Can the team make changes without waiting on many other teams?
- ☐ Does the team have the skills needed to build, test, deploy, and operate this area?
- ☐ Does the boundary reduce the team's cognitive load?
- ☐ Is ownership clear to everyone inside and outside the team?
- ☐ Can the team be accountable for outcomes, not just components?

2. Check for hidden monoliths

- ☐ Are multiple services still tied to the same database schema?
- ☐ Do teams need one large shared build to release changes?
- ☐ Are components bundled into a single coordinated release?
- ☐ Is there one shared domain model forced across many contexts?
- ☐ Are teams constrained by “one-size-fits-all” technology standards?
- ☐ Are teams colocated or arranged in ways that actually support collaboration?
- ☐ Are microservices still dependent on end-to-end testing before release?

3. Look for good fracture planes

Use these as possible ways to split the system.

Business domain / bounded context

- ☐ Does this boundary map to a clear business capability?
- ☐ Does it have its own language, model, and rules?

- ☐ Can business and technical people understand the boundary the same way?
- ☐ Would splitting here reduce confusion or “lost in translation” problems?

Regulatory compliance

- ☐ Does this part of the system have special compliance, audit, or security rules?
- ☐ Can regulated functionality be isolated from less-regulated areas?
- ☐ Would the split reduce the blast radius of compliance oversight?
- ☐ Would it make auditing, traceability, or access control simpler?

Change cadence

- ☐ Does this part of the system need to change more often than others?
- ☐ Is the whole system currently moving at the speed of the slowest part?
- ☐ Would splitting here let fast-changing areas move independently?
- ☐ Would slower-changing areas become more stable?

Team location

- ☐ Are teams separated by geography, time zones, floors, or communication patterns?
- ☐ Does the architecture match how people actually collaborate?
- ☐ Would this split reduce handoffs across time zones?
- ☐ Can teams communicate effectively within their own boundary?

Risk profile

- ☐ Do different parts of the system have different levels of business or technical risk?
- ☐ Can high-risk changes be isolated from low-risk areas?
- ☐ Would this reduce the blast radius of failure?
- ☐ Can each subsystem evolve its own release and operational practices?

Performance isolation

- ☐ Does this area experience different load, scale, or availability needs?
- ☐ Are there peaks of demand that do not affect the whole system?
- ☐ Would isolation allow more targeted scaling or failover?
- ☐ Would this reduce cost or operational complexity elsewhere?

Technology

- ☐ Is this technology significantly different from the rest of the system?
- ☐ Does it require specialized knowledge or manual processes?
- ☐ Would a technology-based split reduce cognitive load?
- ☐ Is this split justified by real constraints, not just technical convenience?

User personas

- ☐ Do different user groups rely on different feature sets?
- ☐ Do some users require advanced controls while others need simplicity?
- ☐ Would this split improve customer experience or support?
- ☐ Can the team focus more sharply on a specific user group's needs?

4. Validate the boundary

- ☐ Can the team provide or consume this subsystem as a service?
- ☐ Can the subsystem be tested independently?
- ☐ Can it be deployed independently?
- ☐ Can it be observed and supported independently?
- ☐ Are contracts, APIs, or events clear between teams?
- ☐ Are dependencies minimal and intentional?
- ☐ Does the split improve flow of change?
- ☐ Does the split reduce coordination overhead?
- ☐ Does the split avoid creating a distributed monolith?

5. Watch for boundary red flags

- ☐ Teams still need frequent synchronous coordination to release.
- ☐ A database change requires many teams to change code.
- ☐ Most work requires updates across several services.
- ☐ End-to-end testing is the only way to gain confidence.
- ☐ Teams own technical layers instead of business outcomes.
- ☐ A team owns too many unrelated responsibilities.
- ☐ Standards remove useful team autonomy.
- ☐ The boundary increases handoffs instead of reducing them.

6. Final decision test

Approve the boundary only if most of these are true:

- ☐ The boundary is aligned to a meaningful business or operational concern.

- ☐ One team can own it sustainably.
- ☐ The team can change and release it with minimal external coordination.
- ☐ The split reduces cognitive load.
- ☐ The split improves flow.
- ☐ The split avoids unnecessary coupling.
- ☐ The team can build, run, observe, and evolve the subsystem over time.